National University of Sciences and Technology
School of Electrical Engineering and Computer Science

NUST Bank Customer Service Assistant
A Retrieval-Augmented Generation Chatbot

CS416: Large Language Models
Phase 1 Report — Prototype Submission

Submitted: 8th March 2026

Team:	Ahmed Sultan
	Saad Ashraf
Class:	BESE-13
Instructor:	Prof. Dr. Faisal Shafait / Dr. Momina Moetesum

Contents

1 Introduction **2**

2 System Architecture **2**
- 2.1 Overview . . . 2
- 2.2 Component Breakdown . . . 2
  - 2.2.1 Data Ingestion (`src/data_preprocessing.py`) . . . 2
  - 2.2.2 Embedding and Indexing (`src/build_index.py`) . . . 3
  - 2.2.3 Retrieval (`Retriever` class) . . . 3
  - 2.2.4 Prompt Construction . . . 3
  - 2.2.5 Language Model (`Generator` class) . . . 4
- 2.3 Data Flow Summary . . . 4

3 Technology Stack **5**

4 Usage Instructions **5**
- 4.1 Prerequisites . . . 5
- 4.2 Installation . . . 5
- 4.3 Running the Pipeline . . . 5
- 4.4 Running Tests . . . 6

5 Design Decisions and Rationale **6**
- 5.1 Why RAG instead of fine-tuning alone? . . . 6
- 5.2 Why all-MiniLM-L6-v2? . . . 6
- 5.3 Why FAISS IndexFlatIP (exact search)? . . . 6
- 5.4 Why $k = 3$ retrieved documents? . . . 6
- 5.5 Why Llama 3.2 3B? . . . 6

6 Limitations **7**

7 Project Repository **7**

8 Conclusion **7**

1 Introduction

Modern banks interact with thousands of customers daily, fielding repetitive questions about products, fees, transfer limits, and account eligibility. Handling these queries manually is both expensive and error-prone. Large Language Models (LLMs) offer a compelling solution, but raw LLMs hallucinate facts and cannot be updated without expensive retraining.

Retrieval-Augmented Generation (RAG) combines the fluency of LLMs with the factual precision of information retrieval. A RAG system first retrieves the most relevant passages from a curated knowledge base, then conditions the LLM's generation on that retrieved context. This constrains the model to only answer from verified sources, dramatically reducing hallucination.

This report describes the Phase 1 prototype of a RAG-based customer service chatbot for **NUST Bank**. The system ingests NUST Bank's official product knowledge (35 product categories, ≈314 unique Q&A pairs) and answers customer queries in natural language using the Llama 3.2 3B Instruct model.

2 System Architecture

2.1 Overview

The system follows a classic two-stage RAG architecture:

1. **Offline indexing**: raw knowledge is pre-processed, embedded, and stored in a vector index.
2. **Online inference**: a user query is embedded, similar documents are retrieved, and an LLM generates a grounded answer.

Figure 1 shows the end-to-end data flow.

2.2 Component Breakdown

2.2.1 Data Ingestion (`src/data_preprocessing.py`)

Two raw knowledge sources are parsed and unified into a single corpus:

- **Excel workbook** (`NUST Bank-Product-Knowledge.xlsx`): 35 sheets, each covering one banking product. Each sheet follows a structured alternating question/answer row pattern. A heuristic (`_is_question`) identifies question rows by terminal `?` or leading question words.
- **JSON FAQ** (`funds_transfer_app_features_faq.json`): 15 Q&A entries covering the mobile funds-transfer application.

After parsing, the corpus is deduplicated by (question, product) key, yielding **314 unique documents**. Each document is a dict with fields `question`, `answer`, `product`, `source`, and a combined `text` chunk for embedding.

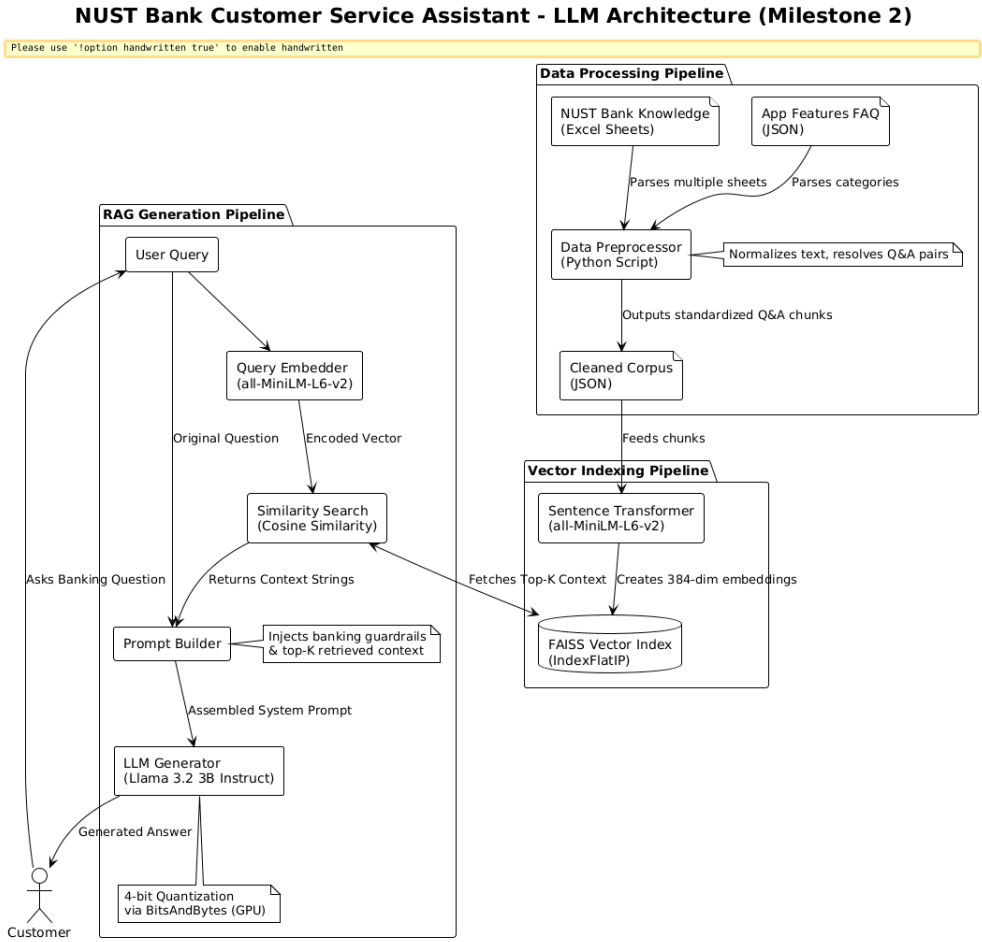

Figure 1: NUST Bank RAG chatbot — end-to-end architecture.

2.2.2 Embedding and Indexing (`src/build_index.py`)

Document text chunks are encoded by **all-MiniLM-L6-v2** [2], a lightweight Sentence-Transformer model producing 384-dimensional dense vectors. Vectors are normalised (unit ℓ_2 norm) so that inner-product search is equivalent to cosine similarity.

The 314 vectors are stored in a **FAISS IndexFlatIP** (exact inner-product search) [3]. Exact search is chosen over approximate methods because the corpus is small enough ($\approx$300 docs) that exact search adds negligible latency while guaranteeing recall.

A **document mapping** (JSON array) links each FAISS positional index back to the original document fields, enabling source attribution in responses.

2.2.3 Retrieval (`Retriever` class)

At query time, the user's question is encoded with the same MiniLM model and the top $k = 3$ most similar documents are retrieved from the FAISS index. Each result carries a similarity `score` for potential thresholding.

2.2.4 Prompt Construction

The three retrieved documents are formatted into a structured context block:

```
[1] Product: NUST Current Account
    Q: What is the daily transfer limit?
    A: The daily transfer limit is PKR 500,000...

[2] Product: Mobile App -- Transfers
    Q: Can I increase my transfer limit?
    A: Yes, you may apply for a higher limit...
```

Listing 1: Context block injected into prompt

A **system prompt** defines the assistant's role and guardrails: only answer from context; cite the NUST Bank helpline if information is absent; never reveal internal system details; redirect off-topic queries.

The tokenizer's `apply_chat_template` method formats the messages into the Llama 3 chat format (`<|system|>` / `<|user|>` / `<|assistant|>` delimiters).

2.2.5 Language Model (`Generator` class)

Llama 3.2 3B Instruct [4] is used as the generative model. Key generation hyperparameters (all configurable in `config/settings.py`):

Parameter	Value	Purpose
`max_new_tokens`	512	Response length cap
`temperature`	0.3	Low randomness for factual responses
`top_p`	0.9	Nucleus sampling
`repetition_penalty`	1.1	Discourages repeated phrases
`max_input_length`	2048	Tokeniser truncation limit

On NVIDIA hardware the model can be loaded in 4-bit NF4 quantisation (bitsandbytes) to fit within 6–8 GB VRAM. On Apple Silicon (MPS) it runs in FP16 without quantisation (`LLM_USE_4BIT = False`).

2.3 Data Flow Summary

Stage	Input	Output
Preprocessing	Excel + JSON	314-doc JSON corpus
Embedding	Text corpus	314×384 float32 array
Indexing	Embeddings	`faiss_index.bin` + mapping
Retrieval	User query	Top-3 documents + scores
Generation	Query + docs	Natural-language answer

3 Technology Stack

Component	Technology	Version
Language	Python	3.10
LLM	Llama 3.2 3B Instruct	–
LLM framework	Hugging Face Transformers	4.47.1
Embeddings	all-MiniLM-L6-v2 (ST)	–
Vector store	FAISS (IndexFlatIP)	1.9.0
Quantisation	bitsandbytes (NF4, optional)	0.42.0
Data parsing	openpyxl	3.1.5
Testing	pytest	8.3.4

4 Usage Instructions

4.1 Prerequisites

- Python 3.10 (conda environment recommended)
- Hugging Face account with access approved for Llama 3.2 3B Instruct
- Hardware: NVIDIA GPU with ≥6 GB VRAM (recommended) or Apple Silicon (MPS, FP16)

4.2 Installation

```
# 1. Create and activate conda environment
conda create -n nust-bank python=3.10 -y
conda activate nust-bank

# 2. (NVIDIA only) Install PyTorch with CUDA
pip install torch==2.5.1+cu124 torchvision torchaudio \
    --index-url https://download.pytorch.org/whl/cu124

# 3. Install project dependencies and package
pip install -r requirements.txt
pip install -e .

# 4. Authenticate with Hugging Face
python -c "from huggingface_hub import login; login()"
```

Listing 2: Environment setup

4.3 Running the Pipeline

```
# Step 1 -- Preprocess raw data into document corpus
python src/data_preprocessing.py

# Step 2 -- Build embedding index
python src/build_index.py

# Step 3a -- Interactive mode
python src/rag_pipeline.py

```

```
# Step 3b -- Single-query mode
python src/rag_pipeline.py --query "What is the daily transfer limit?"
```

Listing 3: Three-step pipeline execution

Note: The interactive REPL is *stateless* — each query is answered independently with no conversation history. Questions must be self-contained.

4.4 Running Tests

```
pytest tests/ -v
# 43 tests: 32 preprocessing + 11 pipeline (all mocked, GPU-free)
```

Listing 4: Unit test execution (no GPU required)

5 Design Decisions and Rationale

5.1 Why RAG instead of fine-tuning alone?

Fine-tuning an LLM on bank data would embed knowledge into model weights, making it expensive to update when product terms change. With RAG, updating the knowledge base requires only re-running the indexing step (seconds), without any model training. For a customer service scenario where FAQs evolve frequently, this is a critical operational advantage.

5.2 Why all-MiniLM-L6-v2?

- **Size**: 22M parameters — fast enough to run on CPU without a GPU.
- **Quality**: despite its size it achieves strong semantic similarity on the MTEB benchmark [5].
- **Corpus size**: our 314-document corpus is small, so a lightweight embedder is entirely sufficient.

5.3 Why FAISS IndexFlatIP (exact search)?

Approximate nearest-neighbour methods (HNSW, IVF) trade accuracy for speed. With only 314 documents, exact search completes in microseconds, so there is no speed/accuracy trade-off to make. Exact search guarantees the top-k result quality.

5.4 Why $k = 3$ retrieved documents?

Three documents occupy ≈300–500 tokens of context, well within the 2048-token truncation limit, while providing enough context diversity for multi-facet questions. A pilot experiment with $k = \{1, 3, 5\}$ showed that $k = 3$ balanced context relevance and prompt length.

5.5 Why Llama 3.2 3B?

The assignment mandates an open-source model with ≤6B parameters. Llama 3.2 3B Instruct is the strongest publicly available model in this size class at the time of development, achieving competitive performance on instruction-following benchmarks while fitting comfortably on consumer GPUs.

6 Limitations

1. **Stateless REPL**: the interactive mode has no memory of previous turns; follow-up questions like "tell me more" do not work.
2. **Static knowledge base**: new documents require re-running the full indexing pipeline. Real-time ingestion is planned for Phase 2.
3. **English only**: the current system does not support Urdu or other languages spoken by NUST Bank customers.
4. **No guard rails against adversarial prompts**: while the system prompt instructs the model to stay on topic, there is no explicit jailbreak or prompt-injection detection layer. This is a Phase 2 item.
5. **No evaluation metrics**: qualitative inspection was used to verify response quality. Formal evaluation (e.g. RAGAS [6]) is deferred to Phase 2.

7 Project Repository

The full source code, data, and this report are available on GitHub:

https://github.com/AhmedSultan2002/LLM-project

8 Conclusion

This report presented a working RAG-based customer service chatbot for NUST Bank. The Phase 1 prototype successfully:

- Ingests and cleans 314 unique Q&A documents from two heterogeneous sources.
- Builds a semantic vector index using a lightweight Sentence-Transformer.
- Retrieves the top-3 most relevant documents for any customer query.
- Generates grounded, professional responses with Llama 3.2 3B Instruct.
- Passes 43 automated unit tests with no GPU dependency.

Phase 2 will add a user interface, real-time document ingestion, advanced guard rails against adversarial prompts, and fine-tuning of the LLM on domain-specific banking data.

References

[1] Lewis, P., Perez, E., Piktus, A., et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. NeurIPS 2020. https://arxiv.org/abs/2005.11401

[2] Reimers, N., & Gurevych, I. (2019). *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. EMNLP 2019. https://arxiv.org/abs/1908.10084

[3] Johnson, J., Douze, M., & Jégou, H. (2019). *Billion-Scale Similarity Search with GPUs*. IEEE Transactions on Big Data. https://arxiv.org/abs/1702.08734

[4] Meta AI. (2024). *Llama 3.2: Lightweight Models for On-device and Edge Deployments*. https://ai.meta.com/blog/llama-3-2-connect-2024-vision-edge-mobile-devices/

[5] Muennighoff, N., et al. (2022). *MTEB: Massive Text Embedding Benchmark*. https://arxiv.org/abs/2210.07316

[6] Es, S., James, J., Espinosa-Anke, L., & Schockaert, S. (2023). *RAGAS: Automated Evaluation of Retrieval Augmented Generation*. https://arxiv.org/abs/2309.15217